

Actividad de Laboratorio: Arquitectura Multi-Nivel y MVC en .NET

Nombre: Diego José Piox Casasola

Carnet:202405843

Curso: Introducción a la programación de computadoras 2.

Sección: P

Fecha:17-06-2026

PARTE 1: Parte Teórica

1. El tránsito hacia los sistemas distribuidos y multi-capa

Limitación del monolito local

Un sistema monolítico local es aquel donde la interfaz de usuario, la lógica de negocio y la base de datos se encuentran instaladas y ejecutándose en una sola computadora. Este enfoque presenta limitaciones importantes en cuanto a escalabilidad, disponibilidad y sincronización de datos. Cuando varios usuarios necesitan acceder a la información, resulta difícil mantener los datos actualizados y consistentes, además de que el rendimiento depende completamente de los recursos de una única máquina. Si el equipo falla, todo el sistema deja de funcionar.

Diferencia entre Layers y Tiers

Aunque los términos suelen confundirse, representan conceptos diferentes.

Layers (Capas): Son divisiones lógicas dentro del software que organizan el código según sus responsabilidades. Por ejemplo, capa de presentación, capa de negocio y capa de datos.

Tiers (Niveles): Son divisiones físicas de la infraestructura donde se ejecutan las aplicaciones. Cada nivel puede estar ubicado en servidores diferentes conectados mediante una red.

En resumen, las capas organizan el software de manera lógica, mientras que los niveles organizan la distribución física de los componentes.

Arquitectura de 3 niveles

Nivel 1: Capa de Presentación (Presentation Tier)

Es la parte del sistema con la que interactúa el usuario. Su función es mostrar información y capturar datos ingresados por el usuario. Generalmente utiliza tecnologías como HTML, CSS, JavaScript y las vistas de ASP.NET Core MVC.

Nivel 2: Capa de Aplicación o Negocio (Application Tier)

Contiene las reglas de negocio y el procesamiento de la información. Se encarga de validar datos, ejecutar procesos y coordinar la comunicación entre la presentación y la base de datos. En aplicaciones .NET suele implementarse mediante controladores, servicios y clases de negocio.

Nivel 3: Capa de Datos (Data Tier)

Es responsable del almacenamiento y recuperación de la información. Aquí se encuentran las bases de datos y los sistemas gestores de datos como SQL Server, MySQL o PostgreSQL. Su función principal es garantizar la persistencia y seguridad de la información.

Seguridad perimetral

Exponer directamente una base de datos a Internet representa un riesgo grave de seguridad, ya que un atacante podría intentar acceder, modificar o eliminar información sensible. La buena práctica consiste en mantener la base de datos dentro de una red privada y permitir el acceso únicamente a través de la capa de aplicación, la cual controla la autenticación, autorización y validación de las solicitudes. Esto reduce significativamente la superficie de ataque y mejora la protección de los datos.

2. Desacoplamiento lógico con el patrón MVC

Código espagueti

El término "código espagueti" se utiliza para describir programas donde la lógica de negocio, las consultas a bases de datos y la interfaz gráfica se encuentran mezcladas en un mismo archivo. Esta situación dificulta la comprensión, mantenimiento y reutilización del código. Además, complica la realización de pruebas unitarias y aumenta la probabilidad de introducir errores al realizar modificaciones.

Separación de preocupaciones (Separation of Concerns - SoC)

El patrón MVC aplica el principio de separación de responsabilidades dividiendo la aplicación en tres componentes principales.

Modelo (Model)

Representa los datos y las reglas de negocio de la aplicación. Su responsabilidad es gestionar la información y las operaciones relacionadas con ella. El modelo no debe conocer cómo se presentan los datos al usuario, ya que su función se limita a administrar la lógica y el estado de la información.

Vista (View)

Es la encargada de mostrar la información al usuario. Se considera una entidad pasiva porque únicamente presenta datos y recibe acciones del usuario. La vista no debe contener reglas de negocio, cálculos complejos ni acceso directo a bases de datos.

Controlador (Controller)

Actúa como intermediario entre el modelo y la vista. Recibe las solicitudes del usuario, coordina las operaciones necesarias, obtiene datos del modelo y selecciona la vista adecuada para responder. Su función principal es dirigir el flujo de la aplicación sin contener lógica de negocio excesiva.

Alta cohesión y bajo acoplamiento

El patrón MVC favorece una alta cohesión porque cada componente tiene responsabilidades claramente definidas y relacionadas entre sí. Al mismo tiempo, promueve un bajo acoplamiento porque los componentes interactúan mediante interfaces bien establecidas, reduciendo las dependencias directas. Esto facilita el mantenimiento, las pruebas, la reutilización del código y la escalabilidad de las aplicaciones profesionales.

Parte 2. Modelado del Ciclo de Vida y Enrutamiento Semántico

URL Entrante	Controlador	Acción	Parámetro ID
https://ingenieria.usac.edu.gt/ControlAcademico/Login	ControlAcademicoController	Login	Ninguno
https://ingenieria.usac.edu.gt/Estudiante/Historial/20260123	EstudianteController	Historial	20260123
https://ingenieria.usac.edu.gt/Asignacion/Detalle/10	AsignacionController	Detalle	10
https://ingenieria.usac.edu.gt/Home	HomeController	Index	Ninguno

ASP.NET Core utiliza la plantilla de enrutamiento {controller=Home}/{action=Index}/{id?}. El primer segmento de la URL identifica el controlador, el segundo la acción que se ejecutará y el tercero corresponde al parámetro opcional id. Cuando no se especifica una acción, se ejecuta Index por defecto.

2. Diagramación del Flujo Interactivo

Flujo de una petición HTTP en MVC

Paso 1: Acción del usuario

El usuario interactúa con la aplicación web realizando una acción, como hacer clic en un botón o seleccionar un enlace desde su navegador. Esta acción genera una solicitud HTTP dirigida al servidor.

Paso 2: Recepción de la petición

La solicitud HTTP llega al servidor donde se encuentra alojada la aplicación ASP.NET Core. El servidor recibe la URL solicitada y la envía al sistema de enrutamiento para su procesamiento.

Paso 3: Enrutamiento de la solicitud

El motor de enrutamiento analiza la URL utilizando la plantilla configurada. Con base en la ruta recibida, identifica qué controlador y qué acción deben ejecutarse para responder a la petición del usuario.

Paso 4: Procesamiento por el controlador

El controlador recibe la solicitud y actúa como intermediario. Si necesita información, consulta al modelo para obtener o procesar datos. Posteriormente prepara la información necesaria y la envía a la vista correspondiente.

Paso 5: Generación y entrega de la respuesta

La vista recibe los datos enviados por el controlador y genera dinámicamente el código HTML. Finalmente, el servidor devuelve la respuesta al navegador, donde el usuario puede visualizar la página web actualizada.

3. Código y funcionalidades

*** Ver repositorio en github**

Parte 4: Auditoría y Control de Calidad

1. Prueba de Cohesión (GET)

La acción `Listar()` devuelve la información de los estudiantes almacenados en memoria y la envía directamente a la vista para su presentación. El controlador se limita a coordinar el flujo de la información sin realizar cálculos complejos ni incluir consultas SQL, cumpliendo así con el principio de responsabilidad única.

2. Evaluación de Antipatrones

Se verificó que los métodos del controlador mantienen una cantidad reducida de líneas de código y cumplen con el principio de "Skinny Controllers". La lógica de procesamiento es mínima y se encuentra separada de la presentación, evitando el antipatrón conocido como "Fat Controller". Esto facilita el mantenimiento, la escalabilidad y la comprensión del sistema.

Referencias Bibliográficas

Facultad de Ingeniería, USAC. (2026). *Sesión 11: Modelado Base y Arquitecturas de Despliegue. Evolución de Sistemas Distribuidos, Fundamentos del Modelo Cliente-Servidor y Diseño Físico Multi-Capas (N-Tier)*. Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.

Facultad de Ingeniería, USAC. (2026). *Sesión 12: Arquitectura y Componentes del Patrón MVC. Desacoplamiento Lógico de Software, Ciclo de Vida de las Peticiones y Enrutamiento en Aplicaciones Interactivas Modernas*. Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.